Mofography native granular visualization (main tab + PLD / percolation)

Arun Gopalan

June 2, 2026

Contents

1 Overview **2**

2 Prerequisites **2**

3 Setup — imports and paths **2**

4 Segmentation **3**

5 Main visualization (interactive app tab) **4**
5.1 Plotly — one-liner 4
5.2 Three.js — one-liner 4
5.3 Three.js — granular `Figure3JS` chain 4

6 PLD and percolation analysis **5**

7 PLD / percolation visualization **7**
7.1 Three.js — one-liner 7
7.2 Three.js — granular `Figure3JS` chain 7
7.3 Plotly — granular traces (optional) 8

8 Trace API quick reference **9**

9 Notes **9**

1 Overview

Reproduce the **Visualization** tab and **PLD & Percolation** tab from `mgr_cli_py` using native `mofography` APIs only.

Layer	Plotly
Main MOF scene	`show_mof_plotly`
PLD / percolation	`visualize.plot_percolated_cluster_meshes` + `plot_percolation_windows`

Plotly-parity trace names: `plot_surface`, `plot_cell`, `draw_framework` / `plot_framework`, `plot_maxima`, `plot_connections`, `plot_windows`.

Working directory: mofography repo root (so `examples/TestCIFs/...` resolves).

Environment: `mofography` installed (`pip install -e .` or dev env); Rust extension built if using `dgrid_from_atoms_rust`.

2 Prerequisites

- CIF under `examples/TestCIFs/` (default below: `FOTNIN_clean.cif`).
- `ase`, `plotly`, `numpy`, `mofography`.
- Emacs: `C-c C-c` on each block (same `:session mofography-viz` keeps variables).

3 Setup — imports and paths

```
from pathlib import Path

import mofography as mgr
from ase.io import read

from mofography.threejs_viz import Figure3JS
from mofography.workflow import (
    show_mof_plotly,
    show_mof_threejs,
    show_percolation_threejs,
)
from mofography.winding_percolation import (
    compute_percolation_history_winding,
```

```
    get_pore_dia_mgr_winding,
)

REPO = Path(".").resolve()  # run from repo root
OUT = REPO / "examples" / "viz_outputs"
OUT.mkdir(parents=True, exist_ok=True)

CIF = REPO / "examples" / "TestCIFs" / "FOTNIN_clean.cif"
SPACING = 0.25
MASK, H, THRESH = 1.5, 0.2, 1.5

print("CIF:", CIF)
print("OUT:", OUT)
```

4 Segmentation

Same pipeline as the interactive app before plotting.

```
atoms = read(str(CIF))
fw_atoms = mgr.update_with_covalent_bonds(atoms.copy())

atoms = mgr.update_with_covalent_bonds(atoms)
dgrid = mgr.dgrid_from_atoms_rust(atoms, spacing=SPACING)
seg = mgr.do_periodic_segmentation_rust(
    dgrid,
    atoms,
    spacing=SPACING,
    mask_thickness=MASK,
    h=H,
    threshold_abs=THRESH,
    return_dgrid=True,
)

stem = CIF.stem
print("seg keys:", sorted(seg.keys()))
print("dgrid shape:", seg["dgrid"].shape)
```

5 Main visualization (interactive app tab)

5.1 Plotly — one-liner

```
fig = show_mof_plotly(seg, atoms, framework_atoms=fw_atoms, stylize=True)
out_plotly = OUT / f"{stem}_main_plotly.html"
fig.write_html(str(out_plotly))
print("Wrote:", out_plotly)
```

5.2 Three.js — one-liner

```
show_mof_threejs(
    seg,
    atoms,
    framework_atoms=fw_atoms,
    output_html=OUT / f"{stem}_main_threejs.html",
    plot_framework_representation="mesh",  # or "instanced" / "scatter"
    plot_surface=True,
    plot_surface_pbc=True,
    plot_connections=True,
    plot_windows=True,
    plot_maxima=True,
    plot_layout="dark",
)
print("Wrote:", OUT / f"{stem}_main_threejs.html")
```

5.3 Three.js — granular `Figure3JS` chain

Explicit control; each `plot_*` adds legend + opacity slider in the HTML HUD.

```
(
    Figure3JS()
    .plot_framework(fw_atoms, representation="mesh")
    .plot_cell(atoms, color="purple")
    .plot_surface(
        seg["dgrid"],
        seg["regions"],
        atoms,
        isovalue=1.0,
        name="Regions",
```

```
        opacity=1.0,
    )
    .plot_surface(
        seg["dgrid"],
        seg.get("regions_pbc"),
        atoms,
        isovalue=1.0,
        name="Regions (PBC)",
        opacity=1.0,
        trace_id="surface_pbc",
    )
    .plot_connections(
        seg["connections"],
        seg["all_maxima"],
        seg["dgrid"].shape,
        atoms,
        hide_periodic=True,
    )
    .plot_windows(seg, atoms=atoms)
    .plot_maxima(seg, atoms)
    .set_camera(eye=(2.0, 0.0, 1.35))
    .to_html(OUT / f"{stem}_main_granular_threejs.html")
)
print("Wrote:", OUT / f"{stem}_main_granular_threejs.html")
```

6 PLD and percolation analysis

Build the `analysis` dict (same as app `_compute_pld_analysis`).

```
dgrid = seg["dgrid"]
lcd, pld, dfsd, _ = get_pore_dia_mgr_winding(
    mat_atoms=atoms,
    dgrid=dgrid,
    kwargs=dict(
        spacing=SPACING,
        mask_thickness=MASK,
        h=H,
        threshold_abs=THRESH,
        min_distance=1.5,
        extra_padding=5.0,
```

```
    ),
    results=seg,
)
percolation_history, pld_locations, _final = compute_percolation_history_winding(
    seg, step=0.05, expand_components=False
)

unique_regions = seg.get("unique_regions", [])
connections = [
    c for c in seg.get("connections", [])
    if all(x in unique_regions for x in c[0])
]
maxima_raw = seg.get("all_maxima", [])
regions = seg["regions"]
maxima = maxima_raw[: int(regions.max())] if len(maxima_raw) else maxima_raw
maxima_info = seg.get("maxima_info")
maxima_radii = (
    dgrid[tuple(maxima_info[:, 1:4].T)] if maxima_info is not None else None
)
rag = mgr.rag_from_connections(connections, maxima)
if maxima_radii is not None:
    rag = mgr.add_maxima_to_rag(rag, maxima, maxima_radii, regions.shape, atoms)
rag.graph["regions"] = regions
rag.graph["ase_atoms"] = atoms

analysis = {
    "lcd": float(lcd),
    "pld": float(pld),
    "dfsd": float(dfsd),
    "percolation_history": percolation_history,
    "pld_locations": pld_locations,
    "dgrid": dgrid,
    "regions": regions,
    "connections": connections,
    "maxima": maxima,
    "maxima_info": maxima_info,
    "maxima_pbc_dict": seg.get("maxima_pbc_dict", {}),
    "rag": rag,
    "isovalue": 1.0,
}
```

```
probe_r = analysis["percolation_history"][-1][0]
print(f"LCD={lcd:.3f} PLD={pld:.3f} probe_r={probe_r:.3f}")
```

7 PLD / percolation visualization

7.1 Three.js — one-liner

```
show_percolation_threejs(
    analysis,
    atoms,
    probe_radius=float(probe_r),
    output_html=OUT / f"{stem}_pld_percolation_threejs.html",
    layout="dark",
)
print("Wrote:", OUT / f"{stem}_pld_percolation_threejs.html")
```

7.2 Three.js — granular `Figure3JS` chain

```
(
    Figure3JS()
    .plot_cell(atoms)
    .plot_connections(
        analysis["connections"],
        analysis["maxima"],
        analysis["dgrid"].shape,
        atoms,
        dgrid=analysis["dgrid"],
        regions=analysis["regions"],
    )
    .plot_maxima(
        analysis["dgrid"],
        analysis["maxima"],
        atoms,
        maxima_info=analysis.get("maxima_info"),
    )
    .plot_percolated_cluster_meshes(
        analysis["percolation_history"],
        analysis["dgrid"],
        analysis["regions"],
```

```
        atoms,
        probe_radius=float(probe_r),
        isovalue=analysis["isovalue"],
    )
    .plot_percolation_windows(analysis, atoms, probe_radius=float(probe_r))
    .to_html(OUT / f"{stem}_pld_granular_threejs.html")
)
print("Wrote:", OUT / f"{stem}_pld_granular_threejs.html")
```

7.3 Plotly — granular traces (optional)

If you prefer Plotly for the PLD tab, assemble traces like the app Plotly branch:

```
import plotly.graph_objects as go
import plotly.express as px
from mofography.visualize import (
    dark_layout,
    plot_cell,
    plot_maxima,
    plot_percolated_cluster_meshes,
    plot_percolation_windows,
    plot_rag,
)

traces = []
traces.append(plot_cell(atoms))
traces.extend(plot_rag(analysis["rag"], name="Pore Graph"))
traces.extend(
    plot_maxima(
        analysis["dgrid"],
        analysis["maxima"],
        atoms,
        maxima_info=analysis.get("maxima_info"),
        maxima_pbc_dict=analysis.get("maxima_pbc_dict"),
    )
)
traces.extend(
    plot_percolated_cluster_meshes(
        analysis["percolation_history"],
```

```
        analysis["dgrid"],
        analysis["regions"],
        atoms,
        probe_radius=float(probe_r),
        isovalue=analysis["isovalue"],
        stepsize=2,
        cscale=px.colors.qualitative.Pastel,
    )
)
traces.extend(
    plot_percolation_windows(analysis, probe_radius=float(probe_r), pld_tolerance=0.25)
)

fig_pld = go.Figure(data=traces)
fig_pld.update_layout(**dark_layout)
out_pld_plotly = OUT / f"{stem}_pld_plotly.html"
fig_pld.write_html(str(out_pld_plotly))
print("Wrote:", out_pld_plotly)
```

8 Trace API quick reference

App layer	Plotly (`visualize.py`)	Three.js functional
Framework mesh	`plot_framework_as_mesh`	`draw_framework(..., representation`
Framework scatter	`draw_framework`	`draw_framework(..., representation`
Unit cell	`plot_cell`	`plot_cell`
Distance surface	`plot_surface_dgrid`	`plot_surface_dgrid`
Maxima	`plot_maxima`	`plot_maxima`
Pore graph	`plot_connections_as_it_is`	`plot_connections`
Windows	`plot_windows`	`plot_windows`
Percolation clusters	`plot_percolated_cluster_meshes`	`plot_percolated_cluster_meshes`
Percolation windows	`plot_percolation_windows`	`plot_percolation_windows`

9 Notes

- Change `CIF` in the Setup block to any structure under `examples/TestCIFs/`.
- HTML files land in `examples/viz_outputs/`; open in a browser (double-click or `open` on macOS).

- To reset the Python session: `M-x org-babel-kill-session RET mofography-viz RET`
- Internal trace IDs (e.g. `percolation_clusters_1_x`) may be renamed in a future API cleanup; Plotly-parity method names stay stable.